

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: E

Batch: 1

Practical No: 5

Roll no: 52

PRN No: 12410371

Name of Student: Vaidik Samarth

Subject Name: Deep Learning

Title of Practical: Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Methodology:

1. Load the IMDB dataset.
2. Limit vocabulary to 10,000 words.
3. Pad/truncate sequences to 200 tokens.
4. Build Simple RNN, LSTM and GRU models.
5. Keep embedding size, units and training configuration identical.
6. Train each model.
7. Evaluate on the held-out test set.
8. Compare Accuracy, Precision, Recall, F1-score, Test Loss and Training Time.
9. Plot validation accuracy.
10. Generate a confusion matrix and classification report for the best model.

Code & Output:

```
1. Import Libraries

import time, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

SEED=42
np.random.seed(SEED); random.seed(SEED); tf.random.set_seed(SEED)
print("tensorflow:", tf.__version__)
```

2. Load the IMDB Dataset

```
VOCAB_SIZE=10000  
MAX_LEN=200
```

```
(X_train,y_train),(X_test,y_test)=tf.keras.datasets.imdb.load_data(num_words=VOCAB_SIZE)  
  
print("training samples:",len(X_train))  
print("testing samples :",len(X_test))  
print("Class distribution:")  
print(pd.Series(y_train).value_counts().sort_index())
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz  
17464789/17464789 ————— 2s 0us/step  
Training samples: 25000  
Testing samples : 25000  
Class distribution:  
0    12500  
1    12500  
Name: count, dtype: int64
```

3. Pad Sequences

```
X_train=tf.keras.utils.pad_sequences(X_train,maxlen=MAX_LEN,padding="post",truncating="post")  
X_test=tf.keras.utils.pad_sequences(X_test,maxlen=MAX_LEN,padding="post",truncating="post")  
print("X_train:",X_train.shape)  
print("X_test :",X_test.shape)
```

```
X_train: (25000, 200)  
X_test : (25000, 200)
```

4. Build RNN, LSTM and GRU Models

```
EMBED_DIM=64  
UNITS=64
```

```
def build_model(layer):  
    model=tf.keras.Sequential([  
        tf.keras.Input(shape=(MAX_LEN,)),  
        tf.keras.layers.Embedding(VOCAB_SIZE,EMBED_DIM),  
        layer(UNITS),  
        tf.keras.layers.Dense(1,activation="sigmoid")  
    ])  
    model.compile(optimizer=tf.keras.optimizers.Adam(0.001),
```

Simple RNN

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 64)	640,000
simple_rnn (SimpleRNN)	(None, 64)	8,256
dense (Dense)	(None, 1)	65

Total params: 648,321 (2.47 MB)

Trainable params: 648,321 (2.47 MB)

Non-trainable params: 0 (0.00 B)

```
> EPOCHS=5
BATCH_SIZE=128
results=[]
histories=[]

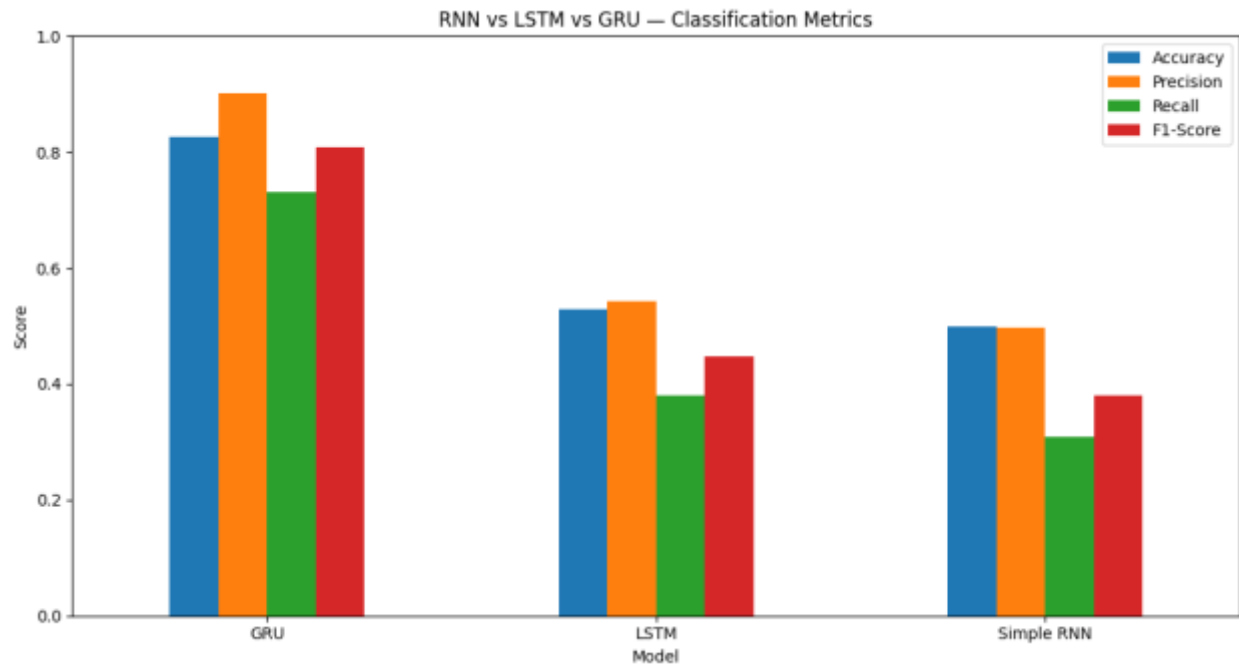
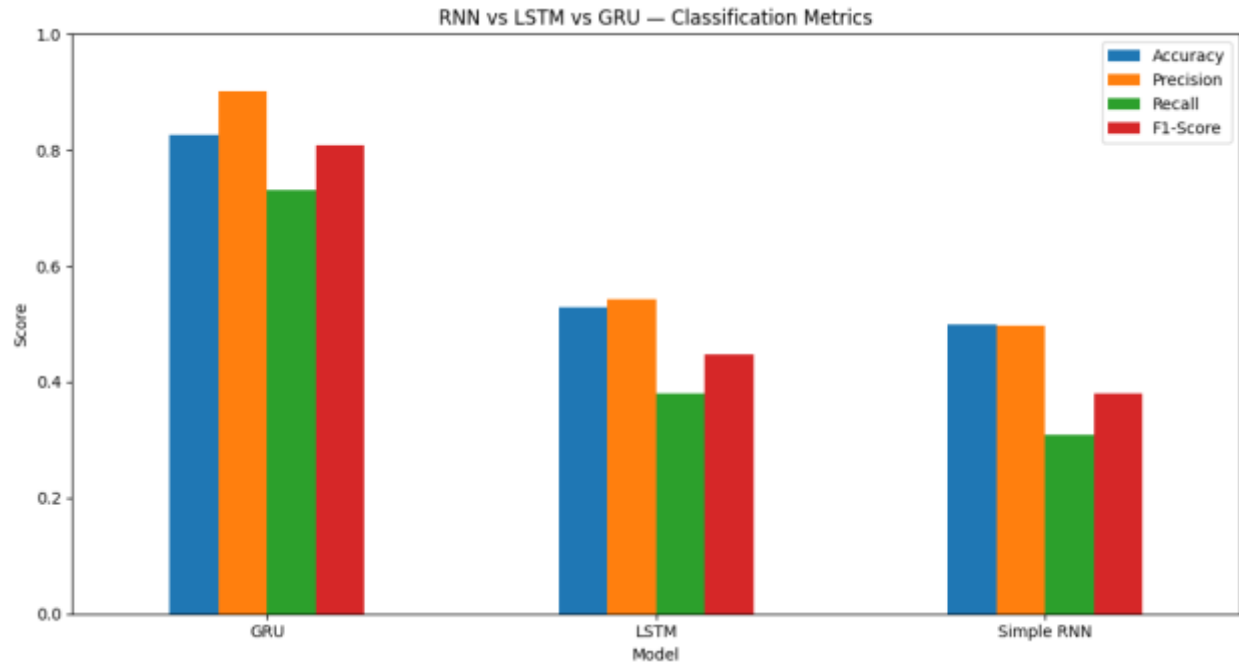
for name,model in models.items():
    print(f"==55, Training:{name}")
    start=time.time()
    history=model.fit(X_train,y_train,validation_split=0.10,
                      epochs=EPOCHS,batch_size=BATCH_SIZE,
                      shuffle=True,verbose=1)
    elapsed=time.time()-start
    histories[name]=history

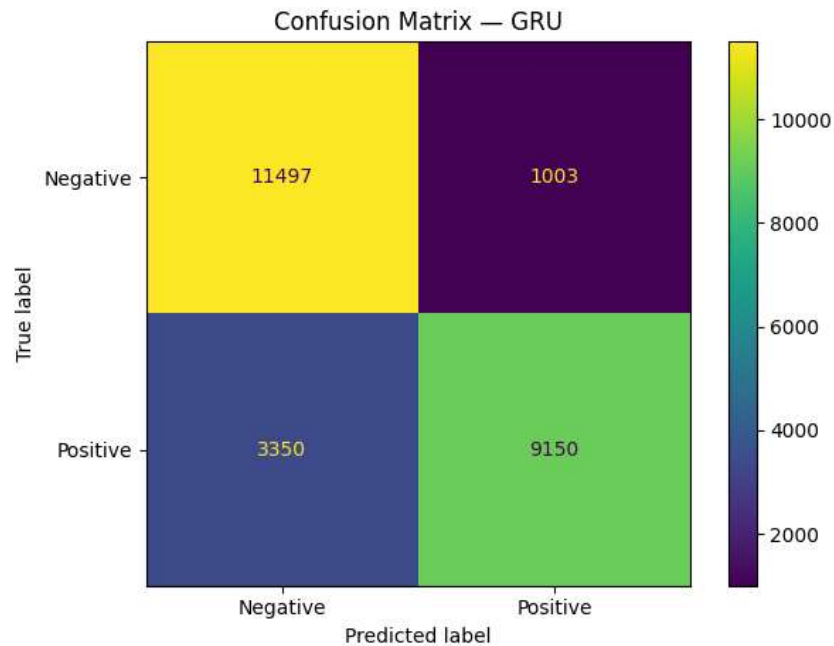
    loss,acc=model.evaluate(X_test,y_test,verbose=0)
    prob=model.predict(X_test,verbose=0).ravel()
    pred=(prob>=0.5).astype(int)

    results.append([
        "Model":name,
        "Test Loss":loss,
        "Accuracy":accuracy_score(y_test,pred),
        "Precision":precision_score(y_test,pred,zero_division=0),
        "Recall":recall_score(y_test,pred,zero_division=0),
        "F1-Score":f1_score(y_test,pred,zero_division=0),
        "Training Time (s)":elapsed
    ])

results_df=pd.DataFrame(results).sort_values("F1-score",ascending=False).reset_index(drop=True)
display(results_df.round(4))
```

[5]





...	precision	recall	f1-score	support
Negative	0.7744	0.9198	0.8408	12500
Positive	0.9012	0.7320	0.8078	12500
accuracy			0.8259	25000
macro avg	0.8378	0.8259	0.8243	25000
weighted avg	0.8378	0.8259	0.8243	25000

9. Final Comparison

```
display(results_df[["Model","Accuracy","Precision","Recall","F1-Score","Test Loss","Training Time (s)"]].round(4))
print("Best model by F1-score:",results_df.iloc[0]["Model"])
```

	Model	Accuracy	Precision	Recall	F1-Score	Test Loss	Training Time (s)
0	GRU	0.8259	0.9012	0.7320	0.8078	0.4001	385.6852
1	LSTM	0.5295	0.5420	0.3806	0.4472	0.6963	335.9691
2	Simple RNN	0.4981	0.4969	0.3086	0.3807	0.9412	101.6962

Best model by F1-score: GRU

Generate Code Markdown

Start Chat to Generate Code (Ctrl+K)

Conclusion:

The experiment compares Simple RNN, LSTM and GRU under the same data and training configuration. Accuracy, Precision, Recall, F1-score, test loss and training time are used together to assess predictive quality and computational cost.

The final winner should be determined from the generated test metrics rather than assumed in advance. Run all cells to obtain the actual results for the execution environment.